



Academy of
Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

THEORY OF COMPUTATION (Course Code: 2304220T)
GROUP MINI-PROJECT ACTIVITY
REGEX-Based Web Form Validation

REPORT ON

Secure File Upload System

MIT Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Alandi (D), Pune– 412105 (2025– 2026)

Student Details

Sr. No	Name	Roll No	PRN
1	Tushar Kaldate	285	202401040191

Name of Faculty/Guide : Dr.Rutuja Kadam

Git Hub: <https://github.com/tusharkkp/regex>

Project Link : <https://tushar-toc-regex.netlify.app/>

Topic: Secure File Upload System (Regex-Based Validation System)

Project Description :

This project implements a Secure File Upload Validation System using purely client-side JavaScript regex patterns backed by formal language theory. The web form accepts seven fields — Username, Email, File Name, File Extension, File Size, Description, and a File Upload widget — and validates each in real-time using carefully designed regular expressions.

Each regex is formally grounded: the accepted language is defined over a finite alphabet, recognisable by a deterministic finite automaton (DFA), and therefore belongs to the class of regular languages as per Kleene's theorem (1956).

The system also demonstrates practical security applications of formal language theory: double-extension attacks, XSS injection, executable uploads, and path-traversal attempts are all blocked by precisely crafted regex patterns using negative lookaheads, character-class restrictions, anchors (^ and \$), and quantifiers.

Objectives :

- To design a web-based form that validates user inputs using Regular Expressions.
- To apply Theory of Computation concepts by representing inputs as regular languages.
- To implement regex patterns for validating fields like username, email, file name, and file type.
- To demonstrate the use of finite automata (DFA/NFA) in pattern recognition and validation.
- To prevent invalid or unsafe file uploads using structured validation rules.
- To provide real-time feedback for improving accuracy and user experience.

Technologies Used :

- **HTML5** – For structuring the web-based form and project layout.
- **CSS3** – For designing a responsive and visually appealing user interface.
- **JavaScript (Vanilla JS)** – For implementing real-time validation logic using Regular Expressions.

- **Regular Expressions (Regex)** – For defining validation patterns based on formal language concepts.
- **Finite Automata (DFA/NFA Concepts)** – For theoretical representation and validation of input patterns.

System Features

- Real-time validation of user inputs using Regular Expressions.
- Multi-field form validation (username, email, file name, type, size, description).
- Secure file validation by restricting extensions and preventing unsafe inputs.
- Implementation of advanced regex concepts like anchors, quantifiers, and lookaheads.
- Representation of validation logic using DFA/NFA and formal language concepts.

Team Members & Contribution

Sr. No	Name	Role	Work Description
1	Tushar Kaldate	Team Lead	All

Challenges Faced :

1. Double-extension attack prevention: Constructing a negative lookahead `(?!.*\.\w+\.\w+)` that correctly blocks "malware.pdf.exe" while still accepting valid filenames required careful anchor placement.

2. Regex vs. semantic validation: File size range $(0 < n \leq 100)$ cannot be expressed purely in regex over decimal strings — required combining the structural regex with a JS numeric range check. Justifying that this does not violate regularity was non-trivial.

3. Lookahead theory alignment: Mapping JavaScript's negative lookahead syntax to formal complement-intersection operations required thorough review of closure properties.

4. DFA state minimisation: Designing minimal DFAs (no unreachable states) for the email pattern with its multi-segment structure needed careful transition table construction.

Key Learnings :

1. **Theory meets practical:** Regular expressions in real-world code are direct implementations of finite automata — every regex corresponds to a DFA recognising exactly the same language (Kleene's theorem in action).
 2. **Closure properties are powerful:** Complex validation rules (block bad extensions, reject whitespace-only) can be formally decomposed into intersections and complements of simpler regular languages.
 3. **Security via formal methods:** Precisely defining the accepted language L forces explicit exclusion of attack vectors — regex-based validation is provably correct when the language is correctly specified.
 4. **Lookaheads are syntactic sugar:** Negative lookaheads do not exceed the power of regular languages — they encode complement+intersection, which are closure-preserving operations on regular languages.
 5. **Pumping Lemma confirmation:** None of our patterns require counting beyond a fixed bound — confirming the Pumping Lemma holds and all languages are genuinely regular.
-

Conclusion

The Secure File Upload Validation System successfully demonstrates the practical application of Regular Expressions in validating user inputs within a web-based form. Through this project, concepts of Theory of Computation such as regular languages and finite automata are effectively applied to real-world scenarios. The system ensures secure and accurate input handling by restricting invalid and unsafe data using well-defined regex patterns. Additionally, the integration of DFA/NFA representations strengthens the theoretical foundation of the project. Overall, this project bridges the gap between formal language theory and modern web development, highlighting the importance of regex-based validation in building reliable and secure applications.